Logging and Error Handling Proposals

- Intro
- 1. What Should We Log?
  - Log Impure Actions, Not Pure Logic
- 2. Where Should Logging Live?
  - Decorating a Typical Service
  - Moving Toward a More Uniform Shape
  - Why This Can Be Helpful
- 3. A Unified Error Model (Error Taxonomy)

  - Proposed Categories

  - Why Bound the Set?

  - ValidationError – The Caller Can Fix It

  - BusinessRuleError – Domain Says No

  - DataIntegrityError – Our Data Is Invalid

  - IoError – Infrastructure Failure
- 4. Semantic Logging + Structured Errors

  - Generic Kill Switch (Cross-Service)
- 5. From Operational Concerns to Code Practices
- 6. Parse, Don’t Validate
- 7. Result Types – Making Failure Explicit

  - Composing Results
- 8. Smart Constructors – Making Invalid States Unrepresentable
- Closing Thoughts (For Discussion)

These are principles we've applied successfully in the past, and they can be adopted incrementally over time. The goal is simply to make our day-to-day support and operational experience a little smoother.

Broadly, this is about:

- Logging in a way that helps us reproduce issues
- Keeping domain logic clean and focused
- Agreeing on a small, consistent set of error types
- Using types to prevent invalid states
- Making failures explicit and observable

Let's walk through this step by step.

1. What Should We Log?

Before we get into how we should log, it's probably worth aligning on what we should log.

If we log too much, we quickly exhaust our logging budget and end up wading through diluted, low-signal noise. If we log too little, we lack the context needed to properly troubleshoot issues when they arise.

Rather than logging everything by default, we could adopt a simple guiding principle:

> Log just enough information to enable repeatable execution.

This is heavily inspired by an excellent post by Mark Seeman - Repeatable Execution

In essence, if we can capture:

- The inputs to our system, and

- The responses from any external dependencies

  then we can usually reproduce the issue with a high degree of confidence.

Log Impure Actions, Not Pure Logic

To make this more concrete, consider a card activation flow:

```
bool canActivate = card.CanBeActivated(currentStatus, expiryDate);
```

This is pure domain logic (or a pure function, for the functional programming enthusiasts among us). Given the same inputs — currentStatus and expiryDate — it will always produce the same output.

If we know the inputs, we can recompute the result. There's little value in logging canActivate.

Now compare that with:

```
var arrangement = await _cardArrangementRepository.Get(cardId);
```

This is an impure (or a non-deterministic) operation. It depends on external state — in this case, the database. If something goes wrong, this is exactly the kind of interaction we'll want visibility into.

For example:

- What card ID was requested?
- What was returned?
- Was the result null?
- Did the returned data violate domain invariants?

So the working principle becomes:

- Log the inputs and outputs of impure functions — typically I/O operations such as:
  - Database calls
  - HTTP calls
  - Messages sent to or received from a broker (RabbitMQ, Kafka, etc.)
- Avoid logging pure domain decisions.

This approach keeps our logs focused and meaningful, giving us the context we need for troubleshooting — without overwhelming ourselves with unnecessary noise.

2. Where Should Logging Live?

Now that we've looked at *what* to log, the next question is:

> Where should logging actually live?

If we place `_logger.LogInformation()` directly inside our services, business logic and diagnostic logic can quickly become intertwined. Over time, this tends to create noise and inconsistency.

An alternative we might consider is treating logging as a **cross-cutting concern** — something applied around our logic rather than embedded within it.

Let's start with a familiar shape.

Decorating a Typical Service

Suppose we have a `CardPanService` exposing three operations:

- `UpdateExpiry`
- `ReplacePan`
- `GetCardTokensByPan`

```
public interface ICardPanService
{
    Task<UpdateExpiryResponse> UpdateExpiry(UpdateExpiryRequest request, CancellationToken ct);
    Task<ReplacePanResponse> ReplacePan(ReplacePanRequest request, CancellationToken ct);
    Task<CardTokensResponse> GetCardTokensByPan(string pan, CancellationToken ct);
}
```

Instead of logging inside the implementation, we can wrap it:

```
public class LoggingCardPanService : ICardPanService
{
    private readonly ICardPanService _inner;
    private readonly ILogger _logger;

    public LoggingCardPanService(ICardPanService inner, ILogger logger)
    {
        _inner = inner;
        _logger = logger;
    }

    public async Task<UpdateExpiryResponse> UpdateExpiry(
        UpdateExpiryRequest request, CancellationToken ct)
    {
        _logger.LogInformation("Updating expiry for {@Request}", request);

        var result = await _inner.UpdateExpiry(request, ct);

```

```
        _logger.LogInformation("UpdateExpiry completed with {@Result}", result);

        return result;
    }

    public async Task<ReplacePanResponse> ReplacePan(
        ReplacePanRequest request, CancellationToken ct)
    {
        _logger.LogInformation("Replacing PAN with {@Request}", request);

        var result = await _inner.ReplacePan(request, ct);

        _logger.LogInformation("ReplacePan completed with {@Result}", result);

        return result;
    }

    public async Task<CardTokensResponse> GetCardTokensByPan(
        string pan, CancellationToken ct)
    {
        // same as above
    }
}
```

This keeps the core `CardPanService` focused purely on card rules, while logging happens consistently at the boundary.

You may notice, however, that each method follows the same pattern:

1. Log request

2. Call inner
3. Log result

That repetition naturally prompts the next question: can we generalise this pattern within the service — and perhaps even make it reusable across other services as well?

Moving Toward a More Uniform Shape

If we model each operation as either a **command** or a **query**, we introduce a consistent method signature — which in turn makes it straightforward to implement a reusable, generic decorator.

```
public interface ICommandHandler<TCommand, TResult>
{
    Task<TResult> Handle(TCommand command, CancellationToken ct);
}

public interface IQueryHandler<TQuery, TResult>
{
    Task<TResult> Handle(TQuery query, CancellationToken ct);
}
```

Now `UpdateExpiry`, `ReplacePan`, and `GetCardTokensByPan` become handlers with the same `Handle` method.

Because the shape is consistent, we can introduce a **single generic decorator**:

```
public class LoggingCommandDecorator<TCommand, TResult>
    : ICommandHandler<TCommand, TResult>
{
    private readonly ICommandHandler<TCommand, TResult> _inner;
```

```
    private readonly ILogger _logger;

    public LoggingCommandDecorator(
        ICommandHandler<TCommand, TResult> inner,
        ILogger logger)
    {
        _inner = inner;
        _logger = logger;
    }

    public async Task<TResult> Handle(
        TCommand command,
        CancellationToken ct)
    {
        _logger.LogInformation("Handling {@Command}", command);

        var result = await _inner.Handle(command, ct);

        _logger.LogInformation(
            "Handled {CommandName} with {@Result}",
            typeof(TCommand).Name,
            result);

        return result;
    }
}
```

Because every command / query shares the same method signature, the decorator becomes reusable across the entire service.

Why This Can Be Helpful

This approach offers a few practical benefits:

- **Separation of concerns** — logging, auditing, caching, metrics, retries can be layered cleanly
- **Consistency** across all operations
- **Avoiding bloated service classes** — by keeping each handler focused on a single responsibility, we reduce the risk of gradually evolving into large, monolithic service classes that accumulate additional functionality and responsibilities over time.
- **Simpler testing** — handlers focus purely on domain behaviour

3. A Unified Error Model (Error Taxonomy)

Now that we've discussed logging and structure, let's talk about errors.

Instead of ad-hoc exceptions and string messages, we could agree on a **bounded set of error types**.

This is not about forcing a rigid structure — it's about agreeing on a small set of categories that meet our domain and operational needs.

The exact model is something we can agree on as a team. What's shown below is illustrative.

Proposed Categories

```
Error
 ├─ ValidationError
 ├─ BusinessRuleError
 ├─ DataIntegrityError
 └─ IoError
```

Why Bound the Set?

This is primarly to avoid error type proliferation.

If we instead agree on a small set that covers:

- Caller mistakes
- Domain rule violations
- Internal corruption
- Infrastructure failures

We gain:

- Better dashboards
- Clearer alerting rules
- Predictable API contracts
- Operational clarity

Let’s look at this through a card domain lens.

ValidationError – The Caller Can Fix It

Example:

- PAN format invalid
- Expiry date malformed
- Required `CustomerId` missing

This is common and expected.

These should:

- Return 400
- Be tracked for trends
- Not trigger alerts

BusinessRuleError – Domain Says No

Example:

- Attempting to activate a card that is already active

- Attempting to suspend a card that is permanently closed
- Issuing a card to a customer flagged as high-risk

The input is valid. The domain rule is violated.

These are not system failures. They are business constraints.

DataIntegrityError – Our Data Is Invalid

This category is more serious.

A `DataIntegrityError` occurs when **data retrieved from our own datastore (or an internal API) violates domain invariants**.

For example, imagine we load a record from the **Card Arrangement DB**:

- The card is marked as `Active`, but the `ProductCode` is missing.
- The `ProductCode` exists but does not match any valid product in **RefDB**.
- Mandatory fields such as `Currency`, `CreditLimit`, or `CustomerId` are null.

In these cases, the caller may have made a perfectly valid request. The failure occurs because **our persisted data is inconsistent or corrupt**.

For example:

```
var dbRecord = await _repository.Get(cardId);
var arrangement = CardArrangement.Restore(dbRecord);

if (arrangement.IsFailure)
{
    return DataIntegrityError.Create(
        operation: "RestoreCardArrangement",
        entityType: "CardArrangement",
        validationFailure: arrangement.Error,
        metadata: new { cardId });
```

```
}
```

This is not a validation issue nor a business rule failure.

This means we were unable to reconstruct a valid domain aggregate from our own data.

A few additional points worth calling out:

- We would expect these errors to be **considerably less frequent** than validation errors.
- They often indicate that data has entered an invalid state — possibly due to:
  - A migration exercise / backfill script
  - Another component writing directly into the datastore and bypassing API-level validation,
  - Or historical data created before certain invariants were introduced.

Because this represents an internal inconsistency, not a caller mistake, it is operationally significant.

In most cases, this would be considered **page-worthy**, as it requires investigation and potentially data repair.

IoError – Infrastructure Failure

An `IoError` represents a failure in one of our external dependencies or infrastructure components.

In our context, examples might include:

- Token service timeout while retrieving card tokens
- RefDB connectivity failure when loading product configuration
- CardArrangementDB timeout during an update
- Message broker publish failure

These are not domain problems / validation issues.

They indicate that something external to our core domain logic failed.

For example:

```
try
{
    var tokens = await _tokenService.GetTokensByPan(pan, ct);
    return Result.Success(tokens);
}
catch (HttpRequestException ex)
{
    return IoError.Create(
        service: "TokenService",
        operation: "GetTokensByPan",
        isRetryable: true,
        metadata: new { MaskedPan: pan.Mask(), ex.Message });
}
```

Or similarly:

```
try
{
    var product = await _refDbClient.GetProduct(productCode, ct);
    return Result.Success(product);
}
catch (TimeoutException ex)
{
    return IoError.Create(
        service: "RefDB",
        operation: "GetProduct",
        isRetryable: true,
```

```
            metadata: new { productCode });
}
```

A few things are worth noting:

- These errors are often **transient** and may be retryable.
- They are operationally significant and should be alertable separately from validation or business rule errors.
- Unlike `DataIntegrityError`, the data itself may be fine — the issue is temporary unavailability or instability of a dependency.

We'll discuss shortly how this pattern could help build a more generic monitoring component (like RCU monitor). The key point here is that strongly typed infrastructure errors give us the foundation to build those kinds of operational safeguards in a consistent way.

4. Semantic Logging + Structured Errors

A small but meaningful shift we can make is towards **semantic (structured) logging**.

Instead of writing free-text log messages, we log structured data alongside a message template. It's a relatively small change in how we write logs, but it has a considerable long-term impact — especially as we transition towards a more modern telemetry stack in the cloud.

For example, instead of:

```
_logger.LogError($"Card activation failed: {error}");
```

We write:

```
_logger.LogError(
```

```
    "Card activation failed with {@Error}",
    error);
```

The `@` tells Serilog to destructure the object, capturing its properties as structured fields rather than flattening everything into a string.

The benefit is immediate:

- We can query by `Error.Type`
- We can group by `Operation`
- We can trend `DataIntegrityError` over time
- We can distinguish `IoError` spikes from validation noise

Even if we are currently using SQL Server as a sink, structured fields already improve queryability. And as we move towards tools like Elastic, Seq, or cloud-native observability platforms, we'll be well aligned — without needing to rethink our logging strategy later.

It's a small change in practice, but one that positions us well for the future.

Generic Kill Switch (Cross-Service)

Building on structured errors:
Instead of building service-specific circuit breakers like an RCU monitor per service, we could build a **generic kill switch component**:

- Monitors error rates by:
  - Error type
  - Operation
  - Dependency
- Can disable:
  - Entire service

- Or specific operations (e.g., Disable `ActivateCard` but allow `GetCardStatus`)

This can gives us reusable operational control across the card ecosystem.

5. From Operational Concerns to Code Practices

Before we move on, just to provide a bit of context — the error taxonomy we discussed earlier is one hierarchy we had put in place at a previous workplace.

It was partly inspired by some Netflix engineering presentations, where they described using a similar structured error model to run and monitor their systems at scale. The idea of having a small, well-defined set of error categories proved extremely helpful from an operational perspective.

That said, the exact hierarchy is not the point. We can absolutely tweak and adapt it to best suit our domain, our card use cases, and our operational needs. The important thing is that we align on a bounded and intentional model rather than letting it evolve organically without structure.

With that foundation in mind, we can now look at some coding practices that help support these operational principles.

So far, we've discussed:

- What to log
- Where logging should live
- How errors can be categorized and observed

If we want operational clarity, the code itself should make invalid states harder to represent and failures easier to reason about. In other words, the way we model our domain can either reinforce — or undermine — the logging and error strategy we've discussed.

That brings us to a few coding patterns that support everything we discussed above.

6. Parse, Don't Validate

There's an excellent article by Alexis King — *Parse, Don't Validate* — that is well worth a read.

The examples are written in Haskell, but the underlying ideas apply just as well in C##.

In our card APIs, we often receive raw input such as:

- PAN
- Expiry date
- CustomerId
- Credit limit

Traditionally, we validate like this:

```
if (!IsValidPan(pan))
    throw new ArgumentException("Invalid PAN");
```

But `pan` is still just a `string`. It's effectively unbounded. The compiler cannot distinguish between a valid PAN and any other arbitrary string.

This is closely related to the **primitive obsession** anti-pattern — where we model important domain concepts using overly generic types like `string`, `int`, or `decimal`.

DDD guidance encourages us to model these concepts as **value objects** instead.

Instead of repeatedly validating a `string`, we parse it once into a more concrete, bounded type:

```
public record Pan
{
    private Pan(string value) => Value = value;

    public string Value { get; }

    public static Result<Pan, ValidationError> Create(string input)
    {
        if (string.IsNullOrWhiteSpace(input))
            return ValidationError.Required("Pan");

        if (!LuhnCheck(input))
            return ValidationError.ValueFormatError("Pan");

        return Result.Success(new Pan(input));
    }
}
```

Here, we are **parsing from an unbounded type (`string`) into a bounded domain type (`Pan`)** — one that must satisfy specific rules (length, format, Luhn check, etc.).

Once constructed:

- If a method accepts `Pan`, it is valid.
- There is no need to re-check.
- Defensive coding reduces significantly.

We perform this parsing at system boundaries:

- API controllers
- Message handlers

- Repository restore points

From that point onward, our domain logic works only with valid, well-defined types — which makes the rest of the system simpler and more reliable.

7. Result Types – Making Failure Explicit

In the examples above, we have introduced a generic `Result<T, TError>` type.

This draws inspiration from languages like F## and Scala where result/union types are built-in.

It's not natively available in C## yet, but trivial to implement one or use one of the many available libraries.

And interestingly, with the union type proposal accepted and expected in C## 15: https://github.com/dotnet/csharplang/issues/9662

We will likely see first-class support in the language.

Instead of hidden exceptions:

```
public Pan Create(string input);
```

We return:

```
public Result<Pan, ValidationError> Create(string input);
```

Failure is now visible at the **method signature level**.

That visibility is important. When we look at a method, we immediately know that this operation can fail and that failure must be handled. It's no longer implicit or hidden in documentation or buried in a potential exception path — the type system makes it explicit.

This also helps prevent exceptions from being used as a form of flow control. Instead:

- Result types model expected, domain-level failures.
- Exceptions are reserved for truly exceptional scenarios — infrastructure failures, programming bugs, or unexpected states.

In our card domain, many failures are expected and part of normal operation:

- Invalid PAN
- Invalid state transitions
- Business rule violations

These aren't exceptional — they're legitimate outcomes that should be handled deliberately and observed appropriately. By modelling them explicitly in the return type, we make that intent clear and keep our exception handling focused on genuinely abnormal conditions.

Additionally, having a strongly typed error model helps us consistently map responses back to clients — including appropriate HTTP status codes and Problem Details specifications — which we will explore shortly.

Composing Results

This approach is composable and can be extended to build multi-field object graph like below:

```
public record CardholderProfile
{
    private CardholderProfile(
        Email email,
        Name name,
        Age age)
```

```
    {
        Email = email;
        Name = name;
        Age = age;
    }

    public Email Email { get; }
    public Name Name { get; }
    public Age Age { get; }

    public static Result<CardholderProfile, ObjectValidationError> Create(
        string email,
        string firstName,
        string lastName,
        int age)
    {
        return ValidationBuilder<CardholderProfile>
            .For(x => x.Email, Email.Create(email))
            .For(x => x.Name, Name.Create(firstName, lastName))
            .For(x => x.Age, Age.Create(age))
            .Build((e, n, a) => new CardholderProfile(e, n, a));
    }
}
```

This gives us a few important advantages:

- **Exhaustive validation rather than fail-fast validation** — instead of stopping at the first failure, we validate all fields and collect every issue.
- **Better client experience** — we can return all validation failures in a single response, rather than forcing the client to fix one error at a time.

- **Cleaner logging** — we can emit one structured log entry containing all validation failures, instead of spreading them across multiple log messages.
- **No partially valid objects** — the object is either fully valid or it does not exist.
- **Guaranteed correctness once constructed** — if a CardholderProfile instance exists, it satisfies all invariants.

8. Smart Constructors – Making Invalid States Unrepresentable

If `CardholderProfile` exists, it should always be valid.

We achieve this by:

- Making the constructor private
- Exposing a static factory method (a “smart constructor”) that validates before creating an instance.

Invalid CardholderProfile never come into existence.

This approach differs from something like FluentValidation, where we typically:

- Create the object
- Then validate it

With that model, the object can temporarily (or accidentally) exist in an invalid state. It also relies on us consistently remembering to invoke the validator after construction.

In a real system, a domain object such as *CardPan* may be accessed through multiple pathways:

- A REST API endpoint
- A message consumed from Kafka
- A nightly batch job

With a post-construction validation approach, we must ensure that every single entry point consistently invokes validation. Missing it in just one pathway can allow invalid state into the system.

By moving validation into the object itself — via a smart constructor — we enforce correctness at the point of creation. There is no separate step to remember. The only way to obtain an instance is through the validated factory method.

In other words:

> The object cannot be created in an invalid state from the outset.

Closing Thoughts (For Discussion)

To recap, we've explored a few practical ideas:

- Log impure boundaries to support repeatable execution
- Use CQRS and decorators to keep logging and metrics cross-cutting
- Agree on a small, bounded error taxonomy
- Leverage structured logging (@ destructuring) for better observability
- Parse at system boundaries into strong domain types
- Use Result types to model expected failures explicitly
- Apply smart constructors to make invalid states unrepresentable

None of this requires a big-bang rewrite. These practices can be introduced gradually, where they add value, and evolve naturally as the codebase grows.